Memory address	Memory content
100	LDD 200
101	STO 202
102	STO 203
103	LDD 202
104	INC ACC
105	STO 202
106	LDD 203
107	ADD 201
108	STO 203
109	CMP 204
110	JPN 103
111	LDD 202
112	OUT
113	END
200	0
201	5
202	
203	
204	75

Figure 6.03 A program to calculate the result of dividing 75 by 5

- The next three instructions are increasing the count by 1 and storing the new value.
- Instructions 106 to 108 add 5 to the sum.
- Instructions 109 and 110 check to see if the sum has reached 75 and if it has not the program begins the next iteration of the loop.
- Instructions 111 to 113 are only used when the sum has reached 75 which causes the value 15 stored for the count to be output.

Shift operations

There are two shift instructions available:

- `LSL #n`
  where the bits in the accumulator are shifted logically n places to the left
- `LSR #n`
  where the bits are shifted to the right.

In a **logical shift** no consideration is given as to what the binary code in the accumulator represents. Because a shift operation moves a bit from the accumulator into the carry bit



KEY TERM

Logical shift: where bits in the accumulator are shifted to the right or to the left and a zero moves into the bit position vacated

in the status register this can be used to examine individual bits. For a left logical shift, the most significant bit is moved to the carry bit, the remaining bits are shifted left and a zero is entered for the least significant bit. For a right logical shift, it is the least significant bit that is moved to the carry bit and a zero is entered for the most significant bit.

If the accumulator content represents an unsigned integer, the left shift operation is a fast way to multiply by two. However, this only gives a correct result if the most significant bit is a zero. For an unsigned integer the right shift represents integer division by two. For example, consider:

00110001 (denary 49) gives if right shifted 00011000 (denary 24)

The remainder from the division can be found in the carry bit. Again, the division will not always give a correct result; continuing right shifts will eventually produce a zero for every bit. It should be apparent that a logical shift cannot be used for multiplication or division by two when a signed integer is stored. This is because the operation may produce a result where the sign of the number has changed.

As indicated earlier, only the two logical shifts are available for the simple processor considered here. However, in more complex processors there is likely to be a **cyclic shift** capability. Here a bit moves off one end into the carry bit then one step later moves in at the other end. All bit values in the original code are retained. Left and right **arithmetic shifts** are also likely to be available. These work in a similar way to logical shifts, but are provided for the multiplication or division of a signed integer by two. The sign bit is always retained following the shift.

KEY TERMS

Cyclic shift: similar to a logical shift but bits shifted from one end reappear at the other end

Arithmetic shift: uses the shift to carry out multiplication or division of a signed integer stored in the accumulator

Bitwise logic operation

The options for this are described in Table 6.10.

Instruction opcode	Instruction operand	Explanation
AND	#Bn	Bitwise AND operation of the contents of ACC with the binary number n
AND	<address>	Bitwise AND operation of the contents of ACC with the contents of <address>
XOR	#Bn	Bitwise XOR operation of the contents of ACC with the binary number n
XOR	<address>	Bitwise XOR operation of the contents of ACC with the contents of <address>
OR	#Bn	Bitwise OR operation of the contents of ACC with the binary number n
OR	<address>	Bitwise OR operation of the contents of ACC with the contents of <address>

Table 6.10 Bitwise logical operation instructions

The operand for a bitwise logic operation instruction is referred to as a mask because it can effectively cover some of the bits and only affect specific bits. Some examples of their use are given in Chapter 7 (Section 7.03).

Question 7.02

Where would you be likely to find a closed-loop feedback control system?

7.03 Bit manipulation to control devices

The controlling computer or microprocessor has to have a real-time program running continuously. The program can set values for Boolean variables subject to what the sensors detect. For instance, if a controlled environment had two properties to be monitored and controlled, four Boolean variables could be used. Values could be set by assignment statements such as:

```
IF SensorDifference1 > 0 THEN Sensor1HighFlag ← TRUE

IF SensorDifference1 < 0 THEN Sensor1LowFlag ← TRUE

IF SensorDifference2 > 0 THEN Sensor2HighFlag ← TRUE

IF SensorDifference2 < 0 THEN Sensor2LowFlag ← TRUE
```

Another part of the monitoring and control program would then be checking whether any of the four flags were set. The machine code for running such a program could use individual bits to represent each flag. The way that flags could be set and read are illustrated by the following assembly language code fragments. In these code fragments the three least significant bits (positions 0, 1 and 2) of the byte are used as flags.

The following illustrates the setting of all bits to zero which might be used when the system is switched on.	
`LDD 0034`	Loads a byte into the accumulator from an address.
`AND #B00000000`	Uses a bitwise AND operation of the contents of the accumulator with the operand to convert each bit to 0.
`STO 0034`	Stores the altered byte in the original address.
The following illustrates the toggling of the value for one bit. This changes the value of the flag it represents. It might be needed because a problem has been encountered or alternatively because a problem has been solved.	
`LDD 0034`	Loads a byte into the accumulator from an address.
`XOR #B00000001`	Uses a bitwise XOR operation of the contents of the accumulator with the operand to toggle the value of the bit stored in position 0.
`STO 0034`	Stores the altered byte in the original address.
The following illustrates the setting of a bit to have value 1 irrespective of its existing value. This would be a simple way of just reporting a condition repetitively.	
`LDD 0034`	Loads a byte into the accumulator from an address.
`OR #B00000100`	Uses a bitwise OR operation of the contents of the accumulator with the operand to set the flag represented by the bit in position 2. All other bit positions remain unchanged.
`STO 0034`	Stores the altered byte in the original address.
The following illustrates setting all bits to zero except one bit which is of interest. Following this operation, a comparison can be made with a binary value to check if the bit is set. In this example the value would be compared to the binary equivalent of denary 2.	
`LDD 0034`	Loads a byte into the accumulator from an address.
`AND #B00000010`	Uses a bitwise AND operation of the contents of the accumulator with the operand to leave the value in position 1 unchanged but to convert every other bit to 0.
`STO 0034`	Stores the altered byte in the original address.

Reflection Point:

Are you clear that a bitwise logic operation acts on every bit individually; in effect all bits in the accumulator are processed simultaneously?

Summary

- A monitoring system requires sensors.
- A sensor measures a physical quantity; there are many examples, such as temperature, humidity, pH, infrared, pressure, sound and carbon monoxide.
- A monitoring and control system requires sensors and actuators.
- A program used for a monitoring and control system has to operate in real time with an infinite loop that accepts input from the sensors at timed intervals.
- The program transmits signals to the actuators if the values received from the sensors indicate a need for control measures to be taken.
- Bit manipulation can be used within an assembly language program to monitor or control devices.

Exam-style Questions

1 A farmer has a large barn to house poultry for the purpose of collecting the eggs that are laid. The environment inside the barn affects the egg-laying performance of the poultry. Traditionally, the farmer had routinely entered the barn to check that all was well with the environment. If there was a concern, the barn had facilities for correcting the problem.

a More recently a computer-based system has been installed. This allows the farmer to observe data on a computer screen. If any of the data is of concern the system has been programmed to show a flashing red sign on the screen.

i Identify the type of system that the farmer has had installed. [1]

ii Identify the type of devices that have been installed inside the barn. [1]

iii Describe **two** examples of this type of device that could be used and explain what their purpose is with respect to the functioning of the computer-based system. [6]

b The farmer has been told that there is no need for someone to be watching a screen all of the time. A different type of computer-based system could be installed.

i Identify the type of this new computer-based system. [1]

ii Identify the new type of device that would need to be installed inside the barn. (There would be more than one needed). [1]

iii Describe how the new computer-based system would interact with these devices. [2]

2 An assembly language program has been written for a monitoring and control system. The program uses a byte stored in a register in which the bits can be individually set or cleared. An example is:

0	1	1	0	0	0	1	0

Bits 0–3 are set to 0 initially but if one of the two sensors in the system sends a measurement that indicates a problem (measurement is too high or too low) the appropriate bit is set to value 1. Bits 4–7 are also set to 0 initially but if an actuator has to be switched on or off the appropriate bit is set to 1.

a All of the bits in the register need to be set to 0. State which logical bitwise operation is required to be performed on the register content and give the operand that would be used for this. Complete your answer by filling in the boxes.

Logical bitwise operation is:

Performed with the operand:

[2]

b A sensor has recorded a value that is too high so bit 2 must be set to 1 but the other bits must remain unaltered. State which logical bitwise operation is required to be performed on the register content and give the operand that would be used for this. Complete your answer by filling in the boxes.

Logical bitwise operation is:

Performed with the operand:

[2]

c Bit 4 is set to 1 and bit 5 set to 0 but the sensor reading now indicates that there has been an over-reaction so the action of the actuator has to be reversed. This requires bits 4 and 5 to have their values toggled. State which logical bitwise operation is required to be performed on the register content and give the operand that would be used for this. Complete your answer by filling in the boxes:

Logical bitwise operation is:

Performed with the operand:

[3]

3 A gardener grows vegetables in a greenhouse. For the vegetables to grow well, the temperature needs to always be within a particular range.

The gardener is not sure about the actual temperatures in the greenhouse during the growing season. The gardener installs some equipment. This records the temperature every hour during the growing season.

a Name the type of system described. [1]

b Identify **three** items of hardware that would be needed to acquire and record the temperature data. Justify your choice for each. [6]

Item 1

Justification

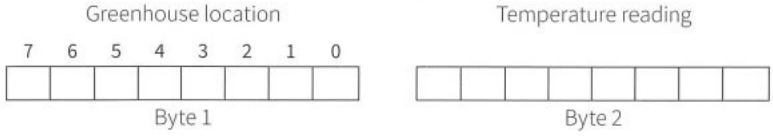
Item 2

Justification

Item 3

Justification

c The equipment records temperatures in the greenhouse. It does this for seven locations. Each recording is stored as two successive bytes. The format is shown below:



The location is indicated by the setting of one of the seven bits in byte 1. For example, location 4 is indicated by setting bit 4.

Bit 0 of byte 1 acts as a flag:

- the initial value is zero
- when the reading has been processed it is set to 1

Byte 2 contains the temperature reading (two's complement integer).

i Interpret the data in byte 1 shown below:



[2]

ii The system receives a temperature reading of –5 degrees from sensor 6.

Complete the boxes below to show the two bytes for this recording. The reading has not yet been processed.



[2]

d **i** The accumulator is loaded with the value of byte 1 from location 106.

Write the assembly language instruction to check whether the reading in byte 2 came from location 4.

```
LDD 106                    // data loaded from address 106
```

[4]

ii Write the assembly language instruction to set the flag (bit 0) of the byte contained in the accumulator to 1. [2]

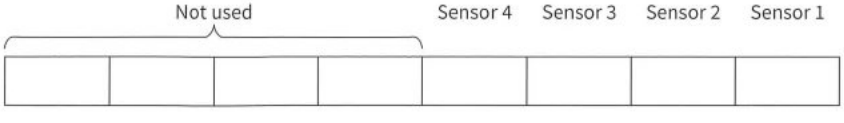
Cambridge International AS & A level Computer Science 9608 Paper 31 Q5 June 2015

4 An intruder detection system for a large house has four sensors. An 8-bit memory location stores the output from each sensor in its own bit position.

The bit value for each sensor shows:

- 1 – the sensor has been triggered
- 0 – the sensor has not been triggered

The bit positions are used as follows:



The output from the intruder detection system is a loud alarm.

a **i** State the name of the type of system to which intruder detection systems belong. [1]

ii Justify your answer to **part (a)(i)**. [1]

b Name **two** sensors that could be used in this intruder detection system. Give a reason for your choice. [4]

The intruder system is set up so that the alarm will only sound if two or more sensors have been triggered.

An assembly language program has been written to process the contents of the memory location.

The table shows part of the instruction set for the processor used.

Instruction		Explanation
Op code	**Operand**	
LDD	<address>	Direct addressing. Load the contents of the given address to ACC.
STO	<address>	Store the contents of ACC at the given address.
INC	<register>	Add 1 to the contents of the register (ACC or IX).
ADD	<address>	Add the contents of the given address to the contents of ACC.
AND	<address>	Bitwise AND operation of the contents of ACC with the contents of <address>.
CMP	#n	Compare the contents of ACC with the number n.
JMP	<address>	Jump to the given address.
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True.
JGT	<address>	Following a compare instruction, jump to <address> if the content of ACC is greater than the number used in the compare instruction.
END		End the program and return to the operating system.

c Part of the assembly code is:

	Op code	Operand
SENSORS:		B00001010
COUNT:		0
VALUE:		1
LOOP:	LDD	SENSORS
	AND	VALUE
	CMP	#0
	JPE	ZERO
	LDD	COUNT
	INC	ACC
	STO	COUNT
ZERO:	LDD	VALUE
	CMP	#8
	JPE	EXIT
	ADD	VALUE
	STO	VALUE
	JMP	LOOP
EXIT:	LDD	COUNT
TEST:	CMP	...
	JGT	ALARM

ii Dry run the assembly language code. Start at `LOOP` and finish when `EXIT` is reached.

BITREG	COUNT	VALUE	ACC
B00001010	0	1	

[4]

ii The operand for the instruction labelled `TEST` is missing.
State the missing operand.

.....[1]

iii The intruder detection system is improved and now has eight sensors.
One instruction in the assembly language code will need to be amended.
Identify this instruction .....
Write the amended instruction.....[2]

Cambridge International AS & A Level Computer Science 9608 paper 31 Q6 June 2016